# DOCKER INTERNALS

*Namespaces · cgroups · Union Filesystems · Linux Inspection Commands*

A Beginner-Friendly Deep Dive with Hands-On Labs

**Namespaces**

Isolation of View

**cgroups**

Resource Limits

**Union FS**

Layer Filesystem

**Commands**

Inspect & Debug

# What is Docker, Really?

*Docker is NOT magic — it is a thin wrapper around 3 core Linux kernel features*

## N

### Namespaces

Isolates what a container can SEE

Each container gets its own private process list, network, filesystem, and hostname.

## C

### cgroups

Limits what a container can USE

Enforces CPU, memory, disk I/O caps per container. Like an electricity meter.

## U

### Union FS

Controls what a container can STORE

Stack of read-only image layers + one thin writable layer per container.

**A container = a process in its own namespace bubble + cgroup limits + a layered filesystem**

# Namespaces

*Isolation of View*

> Analogy: Walls between apartments in a building. Each tenant has their own door number, mailbox, and floor plan — even if the building number is shared.

| PID | NET | MNT |
|---|---|---|
| Private process IDs — container sees PID 1 | Own network stack, IP, routes, iptables | Private filesystem mount tree |
| **UTS** | **IPC** | **USER** |
| Own hostname and domain name | Private inter-process communication | Mapped user IDs for security |

**Host sees:**

```
PID 9821        nginx  (Container A)
PID 9900        mysql  (Container B)
```

**Container A**

```
PID 1 = nginx
IP: 172.17.0.2
/ (own filesystem)
```

**Container B**

```
PID 1 = mysql
IP: 172.17.0.3
/ (own filesystem)
```

**Containers CANNOT see each other's processes or network!**

**Key commands:**

```
lsns                      # list all namespaces
lsns -p <PID>             # container's namespaces
nsenter -t <PID> -n ...   # enter net namespace
ip netns exec ns cmd      # run in named netns
```

# Let’s test your knowledge!!

Virat runs two different web server containers on the same host.
Container A listens on Port 80.
Container B listens on Port 80.
Normally, two applications cannot bind to the same port on the same OS.
Why does Docker allow this without a "Port Conflict" error?

Option 1: **Docker uses Cgroups to split the network card into two physical devices.**

Option 2: **The containers are actually listening on random ports, Virat just thinks it is Port 80.**

Option 3: **Each container runs in its own Network Namespace, giving it a unique IP stack and isolated port range.**

# cgroups

*Control Groups — Resource Limits Per Container*

*Analogy: An electricity meter + circuit breaker per apartment. Each container has a limit on CPU and RAM. Exceed it? The circuit trips (OOMKilled!).*

### cpu

`--cpus=0.5`

Limits how much CPU time this container can use

### memory

`--memory=128m`

Max RAM. If exceeded, process is killed (OOMKilled)

### pids

`--pids-limit=50`

Max number of processes inside the container

### blkio

`--blkio-weight`

Controls disk read/write speed priority

**OOM Kill:** When a container tries to use more RAM than its limit, the Linux kernel's Out-Of-Memory killer sends SIGKILL (exit code 137). Check with: docker inspect <id> --format '{{.State.OOMKilled}}'

**How Docker sets cgroup limits:**

```
# Docker writes to kernel files:
/sys/fs/cgroup/memory/docker/
  <container-id>/
    memory.limit_in_bytes
```

```
# Read the actual limit:
$ cat memory.limit_in_bytes
134217728    # = 128 MB
# 128 * 1024 * 1024 = 134217728
```

```
# Live stats (reads cgroup):
$ docker stats myapp --no-stream
MEM USAGE / LIMIT: 45MiB / 128MiB
```

# Let’s test your knowledge!!

Anushka deploys a new microservice. Due to a memory leak in the code,
the container tries to consume 10GB of RAM, threatening to crash the entire host server.
However, the system miraculously stays stable, and the Linux kernel simply kills that specific container when it hits 1TB.
Which Docker mechanism enforced this safety limit?

Option 1: The network namespace firewall

Option 2: Cgroups

Option 3: Docker deamon

# Let’s test your knowledge!!

According to the Linux kernel features used by Docker, which statement accurately describes the distinction between Namespaces and Cgroups?

Option 1: **Namespaces provide isolation (controlling what a process can SEE), while Cgroups enforce resource limits (controlling what a process can USE).**

Option 2: **Namespaces are used to start the container process (PID 1), while Cgroups are used to stop the container process.**

# Union Filesystem

*OverlayFS — Shared Layers + Thin Writable Layer Per Container*

**Container Layer** — Writable

Writable — YOUR changes go here

**Layer 3: COPY app.py** — Read-only

Read-only — 2 MB

**Layer 2: RUN pip install** — Read-only

Read-only — 45 MB

**Layer 1: FROM python:3.11** — Read-only

Read-only base — 120 MB

## Copy-on-Write (CoW)

When you modify a file in a lower read-only layer, Docker copies the WHOLE file up to the writable layer first, then applies your change. The original is never touched.

```
# Find overlay dirs:
$ docker inspect app \
  --format '{{json .GraphDriver.Data}}'
  UpperDir: .../diff  <- your writes
  LowerDir: .../...   <- image layers
```

### The big disk saving:

100 containers using ubuntu:22.04
= still only ONE copy of the image
on disk. Each adds ~0 bytes!

```
# See image layers:
$ docker history ubuntu:22.04
# See disk usage:
$ docker system df
$ du -sh /var/lib/docker/overlay2
```

# Let’s test your knowledge!!

Sonia is testing a database container.
She runs the container and creates a file at /var/lib/mysql/data.txt.
She stops the container.
She deletes the container (docker rm).
She starts a new container from the same image.
She checks for data.txt and it is gone.
Based on the Union Filesystem architecture, why did the data vanish?

Option 1: **The file was written to the Read-Only Image Layer, which resets on restart.**
Option 2: **The PID Namespace was destroyed, causing the filesystem to unmount.**
Option 3: **She failed to commit the change to the Docker Registry.**
Option 4: **The file was written to the Container's "Writable Layer", which is tied to the container's lifecycle and destroyed when the container is deleted.**

# Let’s test your knowledge!!

Maya is testing a configuration change.
She starts a container from myapp:v1.
She enters the container and modifies /app/config.json.
She stops and removes the container.
She starts a new container from myapp:v1.
She expects to see her modified config, but the file has reverted to its original state. Why?

Option 1: **The Union Filesystem mounted the image as Read-Only, so her writes never actually happened.**
Option 2: **She forgot to run docker commit before stopping the container.**
Option 3: **The modifications were stored in the Writable Container Layer, which is ephemeral and destroyed when the container is removed.**

# Quick Reference Cheatsheet

## Namespace Commands

| | |
|---|---|
| `lsns` | List all active namespaces on host |
| `lsns -p <PID>` | Namespaces of a specific process |
| `nsenter -t PID --all /bin/sh` | Enter all container namespaces |
| `nsenter -t PID -n -- ip addr` | Enter only network namespace |

## Filesystem Commands

| | |
|---|---|
| `mount \| grep overlay` | Show overlay mounts |
| `docker system df` | Docker disk usage overview |
| `du -sh <UpperDir>` | Writable layer size |
| `chroot /tmp/cfs /bin/sh` | Explore exported FS |

## Process Commands

| | |
|---|---|
| `ps auxf` | Full process tree with containers |
| `top -p <PID>` | Monitor container's PID |
| `docker stats --no-stream` | CPU/mem from cgroup files |
| `docker inspect app --format '{{.State.Pid}}'` | Get container's host PID |

## Network Commands

| | |
|---|---|
| `ip addr` | All interfaces inc docker0+veth |
| `ip route` | Routing table (see container subnet) |
| `ss -tulpn \| grep <port>` | Listening ports on host |
| `iptables -t nat -L DOCKER -n` | Docker NAT/port mapping rules |

# Process Inspection Commands

*ps · top · htop · docker stats — Watch containers like a security camera*

## ps

Process Status

Shows container processes with their REAL host PIDs. The f flag draws a process family tree.

```
ps aux | grep nginx
ps auxf | grep -A4 containerd
ps -eo pid,pidns,cmd
```

**Key insight:**

*Container processes ARE real host processes — just in a namespace bubble.*

## top

Real-Time Monitor

Focus top on one container's PID. Press M for memory sort, P for CPU, q to quit.

```
CPID=$(docker inspect app \
  --format '{{.State.Pid}}')
top -p $CPID
```

**Key insight:**

*One container = one high host PID → inside it appears as PID 1.*

## docker stats

Container Dashboard

Reads cgroup files and shows CPU, memory, network I/O in a live updating table.

```
docker stats            # all
containers
docker stats myapp      # one
container
docker stats --no-stream  #
snapshot
```

**Key insight:**

*docker stats reads the same cgroup files the kernel uses for enforcement.*

# Key Takeaways

**N**

### Namespaces = walls

Containers get private PID, NET, MNT, UTS, IPC, USER views. They cannot see each other.

**C**

### cgroups = meters

CPU and RAM limits are stored in /sys/fs/cgroup/ and enforced by the kernel. Exceed = OOMKilled.

**U**

### Union FS = shared cake

100 containers share one image. Each adds only a thin writable layer. Copy-on-write for modifications.

**P**

### Commands = security camera

ps, top, lsns, nsenter, mount, ss — standard Linux tools work on containers from the host.

**Remember: A container is just a process running in a namespace bubble, with cgroup limits, on a layered filesystem.**